

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Department of Information Technology

Fall 2026-27

Operating Systems Lab BITE303P

PROJECT DESIGN & PROGRESS REPORT (REVIEW-I)

KernelLeakGuard: Hybrid Static Analysis and Runtime Verification of Memory Leaks in Linux Kernel Early-Exit Paths

Submitted by

Om Gunjan Gupta 24BIT0316

Abhishek Kumar Singh 24BIT0309

Aarav Jain 24BIT0023

Under the Guidance of

Dr. ARIVUSELVAN K

Associate Professor, SCORE

VIT, Vellore.

Fall 2026-27

CHAPTER 1: SYSTEM ARCHITECTURE

KernelLeakGuard has a static part on the host and a dynamic part inside a QEMU/KVM guest. Static candidates, including the source line of each predicted failing call, drive fault injection in the guest; kmemleak provides the evidence for the verdict.

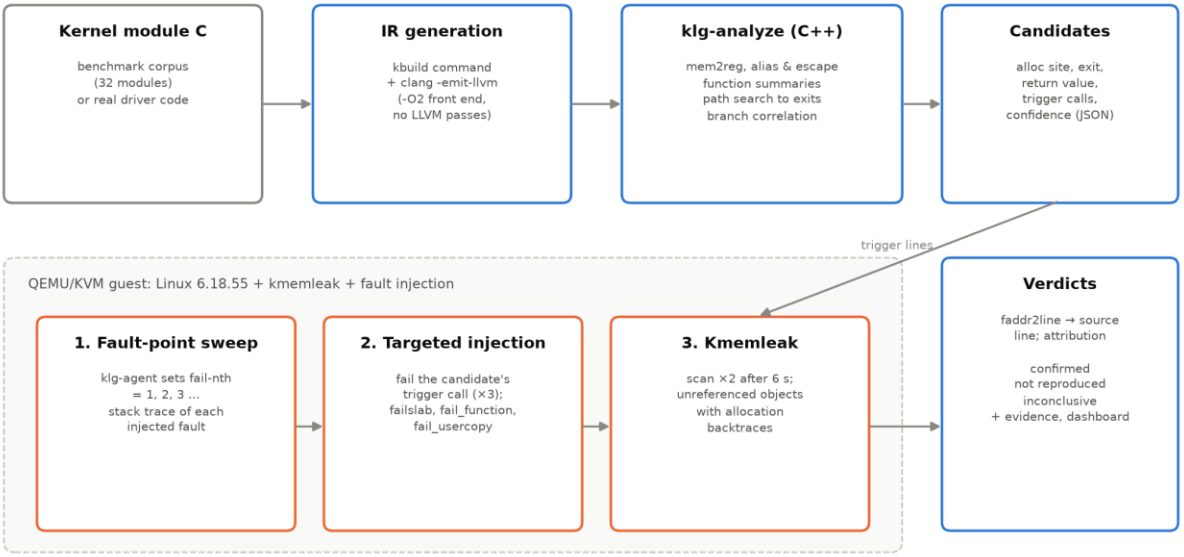


Figure 1.1: System architecture

CHAPTER 2: MODULE DESIGN

Table 2.1: Modules and interfaces

Module	Input	Output	Owner
IR generation (klg/build.py)	module source, kbuild .cmd files	LLVM bitcode (.bc)	Aarav Jain
Static analyser (analyzer/)	bitcode, API specification	candidates JSON	Om Gunjan Gupta
Guest kernel & VM (vm/)	kernel config fragment	bzImage, initramfs, VM runs	Abhishek Kumar Singh
klg-agent (runtime/)	target routine, fault number	attempt log with fault stacks	Abhishek Kumar Singh
Verifier (klg/verify.py)	candidates, agent logs, kmemleak	verdicts with evidence	Aarav Jain
Evaluation & dashboard	verdicts, ground truth	metrics, charts, web UI	Aarav Jain

Data storage. No database is needed. Every stage writes JSON files under results/: static findings per module (results/static/<variant>/<module>.json), verification results with evidence (results/runtime/verification.json) and metrics (results/evaluation.json). A static finding contains the function, allocator, allocation and exit locations, return value, triggers, confidence and path lines.

CHAPTER 3: ALGORITHMS AND FLOWCHARTS

Static leak detection. For each allocation the analyser computes the values that may hold the pointer, marks instructions that release it or transfer ownership, and searches paths to every return. The edge on which the allocation failed and paths that contradict an earlier branch decision are pruned; a path reaching a return without a release or transfer is reported with the failing calls that guard it.

```
for each allocation a in F:
    Alias  := values that may hold a (casts, GEPs, PHIs, stack fields)
    Events := releases (kfree, wrappers) ∪ transfers (non-local store, return, capture)
    DFS from a:  stop at Events; skip "a == NULL" edge; skip contradictory branches
                 at a return → report (alloc, exit, return value, trigger calls)
```

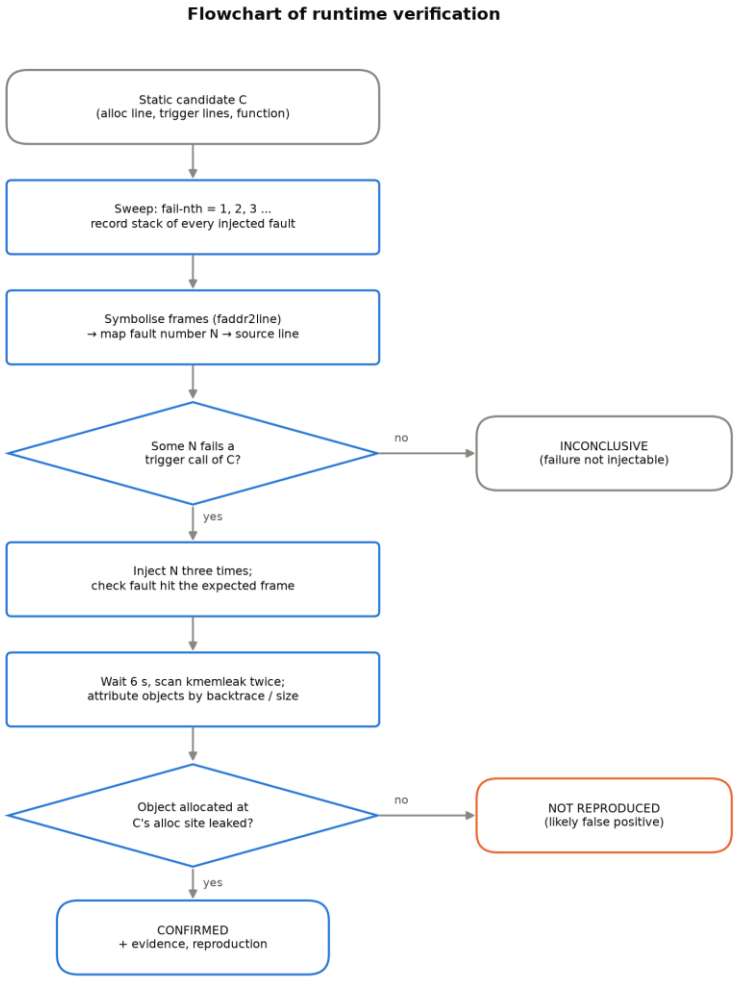


Figure 3.1: Flowchart of runtime verification

CHAPTER 4: IMPLEMENTATION PROGRESS

Table 4.1: Implementation status at Review-I

Component	Status
Kernel 6.18.55 with kmemleak + fault injection, QEMU/KVM boot (~2 s)	Completed
Benchmark corpus: 32 modules with ground truth	Completed
Static analyser: aliases, ownership transfer, summaries, path search, triggers	Completed
klg-agent: fail-nth sweep and targeted repeat	Completed
Fault-point to source-line mapping (faddr2line)	Completed
Targeted confirmation with kmemleak and verdicts	Working prototype
Dynamic baselines, ablation, repeated runs	In progress (Review II)
Dashboard, report, user manual, presentation	In progress (Review II)

The core functionality is executable end to end: `python3 -m klg all` builds the corpus, runs the analyser and verifies the warnings inside the virtual machine.

Screenshots

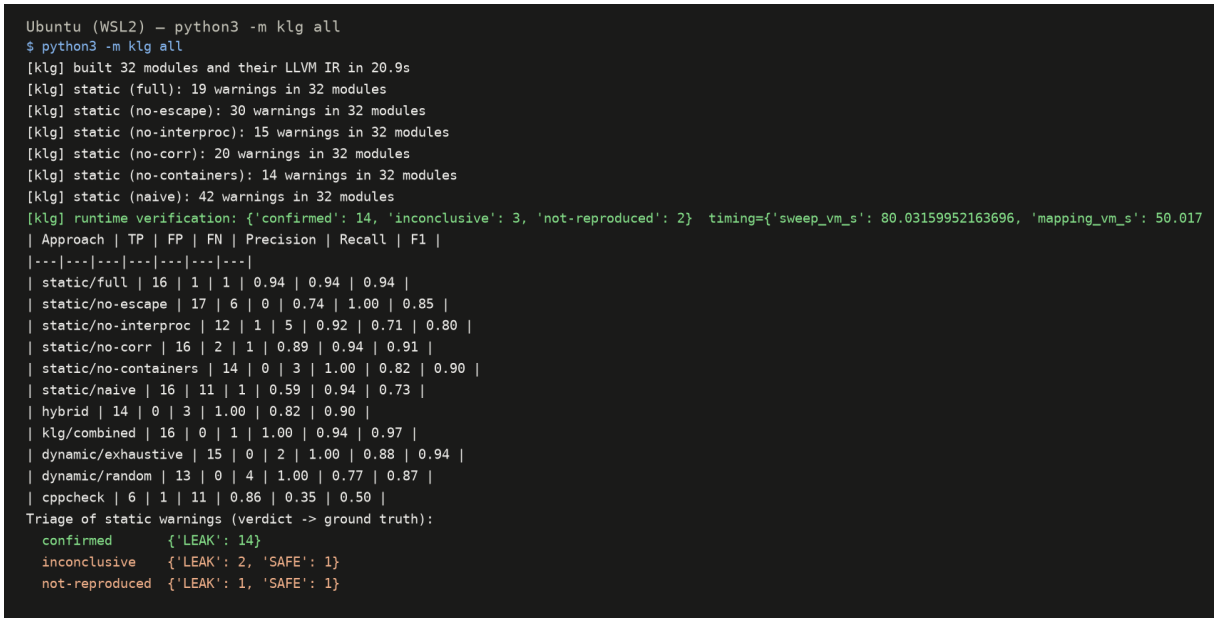


Figure 4.1: Running the complete pipeline

```
Ubuntu (WSL2) — python3 -m klg show
$ python3 -m klg show
klg_l01_two_alloc:14 initialize() kmalloc leaks at line 19 (-ENOMEM, high) -> confirmed
    leak at klg_l01_two_alloc.c:14 after failing initialize+0x46/0xf0
klg_l02_goto_ladder:32 sensor_setup() kmalloc leaks at line 54 (-ENOMEM, high) -> confirmed
    leak at klg_l02_goto_ladder.c:32 after failing sensor_setup+0x5f/0xb0
klg_l03_reg_fail:33 channel_open() kzalloc leaks at line 40 (error from hw_register_channel, high) -> confirmed
    leak at klg_l03_reg_fail.c:33 after failing channel_open+0x32/0x70
klg_l04_loop_partial:19 ring_alloc() kmalloc leaks at line 21 (-ENOMEM, high) -> confirmed
    leak in ring_alloc() after failing ring_alloc+0x48/0x2d0; frame has no line info (merged by optimisation), function's only allocation site is line
klg_l05_usercopy:20 cfg_store() kmalloc leaks at line 25 (-EFAULT, high) -> confirmed
    leak at klg_l05_usercopy.c:20 after failing cfg_store+0x42/0x90
klg_l06_wrapper_alloc:37 session_start() session_create leaks at line 43 (-ENOMEM, high) -> confirmed
    leak at klg_l06_wrapper_alloc.c:37 after failing session_start+0x31/0x80
klg_l07_wrapper_free:27 xfer_run() kzalloc leaks at line 33 (-ENOMEM, high) -> confirmed
    leak at klg_l07_wrapper_free.c:27 after failing xfer_run+0x43/0x90
klg_l08_kstrdup:16 profile_load() kstrdup leaks at line 22 (-ENOMEM, high) -> confirmed
    leak at klg_l08_kstrdup.c:16 after failing profile_load+0x3b/0x60
klg_l09_kcalloc:17 table_build() kmalloc_array leaks at line 34 (-ENOMEM, high) -> confirmed
    leak at klg_l09_kcalloc.c:17 after failing table_build+0x44/0xc0
klg_l10_vmalloc:22 fw_stage() vmalloc leaks at line 28 (-ENOMEM, high) -> confirmed
    leak at klg_l10_vmalloc.c:22 after failing fw_stage+0x37/0x80
klg_l11_kmem_cache:22 req_submit() kmem_cache_alloc leaks at line 28 (-ENOMEM, high) -> confirmed
    leak at klg_l11_kmem_cache.c:22 after failing req_submit+0x3d/0x90
klg_l12_multi_exit:14 packet_prepare() kmalloc leaks at line 27 (-ENOMEM, high) -> confirmed
    leak at klg_l12_multi_exit.c:14 after failing packet_prepare+0x6c/0x160
klg_l13_init_misc:42 ledctl_init() kzalloc leaks at line 49 (error from misc_register, medium) -> confirmed
    leak at klg_l13_init_misc.c:42 after failing init_module+0x3c/0xff0
klg_l14_cond_alloc:29 stats_setup() kzalloc leaks at line 39 (error from stats_calibrate, high) -> confirmed
    leak at klg_l14_cond_alloc.c:29 after failing stats_setup+0x4d/0x80
klg_l15_ida:21 widget_add() kzalloc leaks at line 27 (error from ida_alloc, high) -> inconclusive
    no fault point reaches the failing call (klg_l15_ida.c:25)
klg_l17_local_struct:21 transform_run() kmalloc leaks at line 27 (-ENOMEM, high) -> confirmed
    leak at klg_l17_local_struct.c:21 after failing transform_run+0x42/0x90
klg_s09_loop_cleanup:20 ring_alloc() kmalloc leaks at line 34 (-ENOMEM, high) -> not-reproduced
    predicted failure injected, no object left unreferenced
```

Figure 4.2: Findings with runtime verdicts

```
klg-agent output inside the guest (fault injection)
$ klg-agent repeat --mode trigger --target /sys/kernel/debug/klg_klg_l03_reg_fail --nth 3 --count 3
@@BEGIN n=3
@@ATTEMPT n=3 ret=-1 errno=22 injected=1
K FAULT_INJECTION: forcing a failure.\x0aname fail_function, interval 1, probability 0, space 0, times -1
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? channel_open+0x32/0x70 [klg_l03_reg_fail]
K  ? klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
K  ? hw_register_channel+0x4/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x5/0x20 [klg_l03_reg_fail]
K  hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  channel_open+0x32/0x70 [klg_l03_reg_fail]
K  klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
@@END
@@BEGIN n=3
@@ATTEMPT n=3 ret=-1 errno=22 injected=1
K FAULT_INJECTION: forcing a failure.\x0aname fail_function, interval 1, probability 0, space 0, times -1
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? channel_open+0x32/0x70 [klg_l03_reg_fail]
K  ? klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
K  ? hw_register_channel+0x4/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x5/0x20 [klg_l03_reg_fail]
```

Figure 4.3: klg-agent injecting a failure inside the guest

```
cat /sys/kernel/debug/kmemleak (guest)
$ cat /sys/kernel/debug/kmemleak
unreferenced object 0xff11000005696b08 (size 8):
  comm "klg-agent", pid 163, jiffies 4294734140
  hex dump (first 8 bytes):
    03 00 00 00 00 00 00 00                         ........
  backtrace (crc 8e8f07e3):
    __kmalloc_cache_noprof+0x1b5/0x420
    channel_open+0x1c/0x70 [klg_l03_reg_fail]
    klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
    full_proxy_write+0x5c/0xb0
    vfs_write+0x136/0x450
    ksys_write+0x79/0x100
    do_syscall_64+0xc8/0x260
    entry_SYSCALL_64_after_hwframe+0x77/0x7f
```

Figure 4.4: kmemleak reporting the leaked object

CHAPTER 5: CHALLENGES FACED

Table 5.1: Challenges and solutions

Challenge	Cause	Solution
Every kmalloc() appeared twice in the IR	kmalloc's macro tests mem_alloc_profiling_enabled() even when profiling is off	Fold constant-returning stubs and remove dead branches before analysis
Correlated conditions reported as leaks	Path search started at the allocation and ignored the guarding condition	Seed the search with dominating branch decisions
Loop leak missed	Decisions from iteration 1 applied to iteration 2	Mark decisions stale when a path goes round a loop again
fail_function ignored fail-nth	int3 kprobe handlers run in NMI-like context where in_task() is false	Enable ftrace-based kprobes (KPROBES_ON_FTRACE)
Guest kernel panic in random baseline	Probability raised before task filter, faulting PID 1	Enable task-filter first; agent enables faults only around the target syscall
Fault sites not symbolised	faddr2line rejects frames whose kallsyms size includes padding	Pass func+offset without size
Unrolled loop frames had no source line	Merged debug locations	Fall back to the function's single allocation site
Leak hidden in some runs (klg_l17)	Stale pointer copies on reused stacks and in reused slab objects	KSTACK_ERASE, no task-stack scan, init_on_alloc/init_on_free